Chapter 9 — DDL, DML and Constraints: Building Tables and Changing Data Safely

This chapter creates structures and changes data: **DDL — Data Definition Language** (`CREATE` , `ALTER` , `DROP` , `TRUNCATE` , `CREATE INDEX`); **keys and constraints** (primary and foreign keys, `UNIQUE` , `CHECK` , `NOT NULL` , `DEFAULT`), enforced on every change whichever application writes; **DML — Data Manipulation Language** (`INSERT` , `UPDATE` , `DELETE` , PostgreSQL's `ON CONFLICT` upsert and `RETURNING`); and **backup and restore**, run for real with `pg_dump` / `pg_restore` (command-line programs that write a database to a file and load it back). Table design (entities, relationships, normal forms — rules for keeping each fact in one place) is chapter 10 and chapter 11.

Sections: 9.1 Data types · 9.2 `CREATE` · 9.3 Keys and constraints · 9.4 Generated keys · 9.5 `ALTER` · 9.6 `DROP` and `TRUNCATE` · 9.7 Indexes (brief) · 9.8 `INSERT` · 9.9 `UPDATE` · 9.10 `DELETE` · 9.11 Upsert and `MERGE` · 9.12 Backup and restore.

Setup. The `company` database (chapter 1, 1.11); each file in `code/` starts from a fresh copy. Each `ERROR:` shown is the database deliberately refusing something.

9.1 Data types

What it is

A column's **data type** decides storage, valid operations, sort order and — just as important — which values are **rejected**: the first and cheapest constraint.

The common types

Kind	PostgreSQL type	Holds	Use for	MySQL / SQL Server notes
Integer	`smallint` (2 bytes)	−32 768 … 32 767	small ranges: ranks, ratings	same names
	`integer` / `int` (4 bytes)	about ±2.1 billion	counts, most ids	same
	`bigint` (8 bytes)	about ±9.2 × 10^{18}	large ids, populations, byte counts	same
Exact decimal	`numeric(p, s)` / `decimal(p, s)`	*p* digits in total, *s* after the point	**money**, anything that must add up exactly	same
Approximate	`real`, `double precision`	binary floating point (fast but approximate)	measurements, science, ML (machine-learning) features	`FLOAT` / `DOUBLE`; SQL Server `float`
Fixed text	`char(n)`	exactly *n* characters, padded with spaces	fixed codes (rarely worth it)	same
Variable text	`varchar(n)`	up to *n* characters	text with a real maximum length	SQL Server: `nvarchar(n)` for Unicode (text in any alphabet)
	`text`	any length	most text in PostgreSQL	MySQL `TEXT` has limits and index restrictions
Date / time	`date`	a calendar day	birthdays, due dates	same
	`time`	time of day	opening hours	same
	`timestamp`	date + time, **no** time zone	local "wall clock" times	MySQL `DATETIME`
	`timestamptz`	an exact instant (stored in UTC — the world's reference clock — shown in the session's time zone; a *session* is your current connection)	event times, created_at	MySQL `TIMESTAMP` (converted to UTC); SQL Server `datetimeoffset`
Year only	(none) — use `smallint` + `CHECK`, or `date`			MySQL has `YEAR`
Boolean	`boolean`	true / false / NULL	flags	MySQL `BOOLEAN` = `TINYINT(1)`; SQL Server `bit`
Other useful	`uuid`, `jsonb`, arrays, `inet`, ranges		see chapter 14	

Conversions, casts and per-type functions: chapter 3.

Seeing the rules in action:

```sql
-- code/01_data_types.sql
SET TimeZone = 'UTC';   -- make timestamptz output the same on every machine
CREATE TABLE type_demo (
    qty        integer,
    views      bigint,
    rank_no    smallint,
    price      numeric(10,2),
    ratio      double precision,
    code       char(5),
    name       varchar(10),
    notes      text,
    born_on    date,
    starts_at  time,
    created    timestamp,
    created_tz timestamptz,
    active     boolean,
    birth_year smallint CHECK (birth_year BETWEEN 1900 AND 2100)  -- no YEAR type in PostgreSQL
);
INSERT INTO type_demo VALUES
 (3, 9000000000, 7, 19.999, 0.1::float8 + 0.2::float8, 'AB', 'Tara', 'long text ...',
  '1994-09-22', '09:30', '2025-06-01 10:00', '2025-06-01 10:00+05:30', true, 1994);

SELECT price, ratio, '[' || code || ']' AS code_padded, length(code) AS len_code,
       octet_length(code) AS bytes_code, created, created_tz
FROM type_demo;
```

Output:

```
 price |        ratio        | code_padded | len_code | bytes_code |       created       |       created_tz
-------+---------------------+-------------+----------+------------+---------------------+------------------------
 20.00 | 0.30000000000000004 | [AB]        |        2 |          5 | 2025-06-01 10:00:00 | 2025-06-01 04:30:00+00
(1 row)
```

- `price` 19.999 was **rounded** to 20.00.
- `ratio`: binary 0.1 + 0.2 isn't exactly 0.3 (`0.1::float8` is a **cast** — conversion — to an 8-byte float; `0.1::numeric + 0.2::numeric = 0.3` is true) — why money is never `float`.
- `code` is padded to 5 characters (5 bytes) but `length` ignores a `char`'s trailing spaces and says 2 — confusing; prefer `text` / `varchar`.
- `created_tz`: 10:00 in India (+05:30) became an instant shown in UTC as 04:30. **What the types reject:**

```sql
INSERT INTO type_demo (name) VALUES ('Christopher Columbus');
INSERT INTO type_demo (rank_no) VALUES (40000);
INSERT INTO type_demo (price) VALUES (123456789.00);
INSERT INTO type_demo (born_on) VALUES ('2025-02-30');
INSERT INTO type_demo (birth_year) VALUES (1850);
```

Output:

```
ERROR:  value too long for type character varying(10)
ERROR:  smallint out of range
ERROR:  numeric field overflow
DETAIL:  A field with precision 10, scale 2 must round to an absolute value less than 10^8.
ERROR:  date/time field value out of range: "2025-02-30"
ERROR:  new row for relation "type_demo" violates check constraint "type_demo_birth_year_check"
...
```

`numeric(10,2)` allows 10 digits, 2 after the point, so $< 10^8$. MySQL in non-strict mode (errors become warnings) would silently truncate some of these; PostgreSQL always errors.

How to choose

1. **Money** → `numeric(p, s)` (or integer cents), never `float`. **Ids/counts** → `integer`, or `bigint` if it could ever pass 2 billion.
2. **Text** → `text` in PostgreSQL; `varchar(n)` only when the maximum is a real business rule (no speed difference). SQL Server: `nvarchar` for Unicode.

3. **Instants** → `timestamptz`; `timestamp` only for wall-clock times that must not shift (a shop's opening time). **Days** → `date`.
4. Pick the smallest type that safely fits — changing it later on a big table is expensive (9.5).

Common mistakes

Money in `float` (rounding errors); dates as `text` (`'03/04/2025'` — March or April? can't sort); `varchar(255)` everywhere by habit (a meaningless limit); `timestamp` for events across time zones (ambiguous instants).

9.2 CREATE: databases, schemas and tables

What it is

`CREATE` makes new objects:

Statement	Creates
`CREATE DATABASE name;`	a new, separate database on the server (the running database program)
`CREATE SCHEMA name;`	a namespace **inside** a database (like a folder); tables are then `schema.table`
`CREATE TABLE name (...);`	a table: columns, types, constraints
`CREATE TABLE name AS SELECT ...;`	a table filled with a query's result
`CREATE TABLE name (LIKE other INCLUDING ALL);`	an empty copy of another table's structure
`CREATE TEMP TABLE name (...);`	a table visible only to your session, dropped automatically at the end
`CREATE VIEW`, `CREATE INDEX`, `CREATE SEQUENCE`, `CREATE FUNCTION` …	covered in chapters 7, 9.7, 9.4, 14

How CREATE TABLE works

```
CREATE TABLE [IF NOT EXISTS] [schema.]table_name (
    column_name  data_type  [column constraints],   -- one line per column
    ...,
    [table constraints]                             -- e.g. PRIMARY KEY (a, b)
);
```

(Square brackets mark optional parts.) The name must be unique in its schema (default `public`); each column has a name, type, optional constraints (9.3) and `DEFAULT`; multi-column constraints follow the columns. `IF NOT EXISTS` turns "already exists" into a notice (not an error) — handy in scripts, but it does **not** check the existing table's columns.

```sql
-- code/02_create.sql
CREATE DATABASE school_demo;          -- a new database (then dropped again in the script)
CREATE SCHEMA school;

CREATE TABLE school.students (
    student_id   integer      PRIMARY KEY,
    first_name   varchar(50)  NOT NULL,
    last_name    varchar(50)  NOT NULL,
    roll_number  integer      UNIQUE,
    email        text,
    enrolled_on  date         NOT NULL DEFAULT current_date
);

CREATE TABLE IF NOT EXISTS school.students (student_id integer);

CREATE TABLE school.bad (
    id   integer PRIMARY KEY,
    name text,
);
```

Output:

```
NOTICE:  relation "students" already exists, skipping
ERROR:  syntax error at or near ")"
LINE 4: );
```

A trailing comma after the last column is a syntax error (PostgreSQL, MySQL, SQL Server).

Tables from queries and copies:

```sql
CREATE TABLE school.sales_staff AS
SELECT emp_id, first_name, salary FROM employees WHERE dept_id = 2;
SELECT * FROM school.sales_staff ORDER BY emp_id;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 150000.00
      7 | Priya      |  72000.00
 ...
(4 rows)
```

`CREATE TABLE … AS` copies names, types and data but **no constraints, defaults or indexes** (`SELECT … INTO new_table FROM …` does the same in PostgreSQL and SQL Server). For structure with constraints and defaults, use `LIKE`:

```sql
CREATE TABLE school.products_copy (LIKE products INCLUDING ALL);   -- 0 rows, same columns/constraints
```

Temporary tables (`CREATE TEMP TABLE scratch (id int, note text);`) live in a private per-session schema (the **catalog** — PostgreSQL's internal tables describing objects — shows a `pg_temp_N` schema and `relpersistence = 't'` instead of `'p'`) and vanish at session end, or transaction end with `ON COMMIT DROP`. Use them for multi-step work in one session (7.9).

> **Key term — transaction:** an all-or-nothing group of statements: `BEGIN`, then `COMMIT` (make permanent) or `ROLLBACK` (undo). Without `BEGIN`, each statement is its own transaction (chapter 13).

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Database vs schema	database contains schemas	"schema" = "database" (synonyms)	database contains schemas (`dbo` default)	one file = one database
Switch database	reconnect (`\c db` in psql, PostgreSQL's command-line client)	`USE db;`	`USE db;`	open another file / `ATTACH`
`CREATE TABLE IF NOT EXISTS`	yes	yes	no (check `OBJECT_ID` first)	yes
Temporary table	`CREATE TEMP TABLE`	`CREATE TEMPORARY TABLE`	`CREATE TABLE #name`	`CREATE TEMP TABLE`
`CREATE DATABASE` inside a transaction	not allowed	implicit commit (silently ends the open transaction first)	not allowed	n/a

---

9.3 Keys and constraints

What they are

A **key** is a column (or set) whose values identify rows. A **constraint** is a rule checked on every write; a statement breaking it fails completely. Applications have bugs and there are usually several of them, but every write passes through the database — so the rules belong there.

The kinds of keys

For `members(member_id, email, phone, full_name)`:

Key	Definition	In `members`
Super key	any column set that is unique for every row (may include unnecessary columns)	`{member_id}`, `{member_id, full_name}`, `{email, phone}`, …
Candidate key	a *minimal* super key — drop any column and it stops being unique; a table can have several	`{member_id}`, `{email}`
Primary key (PK)	the candidate key chosen as main identifier: unique **and** `NOT NULL`, one per table	`member_id`
Alternate key	a candidate key that wasn't chosen as primary; enforced with `UNIQUE`	`email`
Unique key	columns declared `UNIQUE`: no duplicates, NULLs allowed (below); many per table	`email`, `phone`
Composite key	two or more columns; only the *combination* must be unique	`(club_id, member_id)` in `club_memberships`
Foreign key (FK)	column(s) whose values must match a key (usually the PK) of another (or the same) table	`club_memberships.member_id` → `members.member_id`
Natural vs surrogate	natural = a real-world value (email, ISBN); surrogate = a generated number/UUID (9.4)	`email` is natural, `member_id` is surrogate

Key theory (functional dependencies): chapter 10, chapter 11.

The constraints

Constraint	Rule	Written as
`NOT NULL`	a value is required	`col type NOT NULL`
`DEFAULT`	value used when an `INSERT` omits the column (not a check, but declared alongside)	`col type DEFAULT expr`
`UNIQUE`	no two rows share the value(s); NULLs don't count as equal	`col type UNIQUE` / `UNIQUE (a, b)`
`PRIMARY KEY`	`UNIQUE` + `NOT NULL`; one per table; creates a unique index (a lookup structure, 9.7)	`col type PRIMARY KEY` / `PRIMARY KEY (a, b)`
`CHECK`	a boolean expression must not be false	`CHECK (price >= 0)` / `CHECK (end_date >= start_date)`
`FOREIGN KEY`	value must exist in the referenced key (or be NULL)	`col type REFERENCES other(col)` / `FOREIGN KEY (a) REFERENCES other(x)`
`EXCLUDE` (PostgreSQL)	no two rows may "conflict" by an operator (e.g. overlapping bookings of one room)	see chapter 14

A **column constraint** sits on the column; a **table constraint** after the columns (required for multi-column rules). Name important ones (`CONSTRAINT name …`) — the name appears in errors and is needed to drop it; otherwise PostgreSQL generates names like `members_email_key`.

```
-- code/03_keys_and_constraints.sql
CREATE TABLE members (
    member_id  integer PRIMARY KEY,                       -- primary key
    email      text    NOT NULL UNIQUE,                   -- candidate key -> alternate key
    phone      text    UNIQUE,                            -- unique key, NULL allowed
    full_name  text    NOT NULL,
    plan       text    NOT NULL DEFAULT 'free'
                       CONSTRAINT plan_is_known CHECK (plan IN ('free', 'pro', 'team')),
    joined_on  date    NOT NULL DEFAULT current_date,
    left_on    date,
    CONSTRAINT left_after_joined CHECK (left_on IS NULL OR left_on >= joined_on)   -- table-level check
);

INSERT INTO members (member_id, email, phone, full_name, joined_on)
VALUES (1, 'ravi@mail.example', NULL, 'Ravi Kumar', '2025-01-10'),
       (2, 'lena@mail.example', NULL, 'Lena Ortiz', '2025-02-01');     -- two NULL phones: fine
SELECT member_id, email, phone, plan FROM members ORDER BY member_id;
```

Output:

```
 member_id |       email       | phone  | plan
-----------+-------------------+--------+------
         1 | ravi@mail.example | (null) | free
         2 | lena@mail.example | (null) | free
(2 rows)
```

`plan` was filled from its `DEFAULT`. Now each rule refusing bad data:

```
INSERT INTO members (member_id, email, full_name) VALUES (1, 'x@mail.example', 'Dup Id');
INSERT INTO members (member_id, email, full_name, joined_on) VALUES (NULL, 'y@mail.example', 'No Id', '2025-03-
01');
INSERT INTO members (member_id, email, full_name) VALUES (3, 'ravi@mail.example', 'Dup');
INSERT INTO members (member_id, email, full_name, plan, joined_on) VALUES (4, 'z@mail.example', 'Z', 'gold',
'2025-03-01');
INSERT INTO members (member_id, email, full_name, joined_on, left_on)
VALUES (5, 'w@mail.example', 'W', '2025-03-01', '2025-01-01');
```

Output:

```
ERROR:  duplicate key value violates unique constraint "members_pkey"
DETAIL:  Key (member_id)=(1) already exists.
ERROR:  null value in column "member_id" of relation "members" violates not-null constraint
...
ERROR:  duplicate key value violates unique constraint "members_email_key"
...
ERROR:  new row for relation "members" violates check constraint "plan_is_known"
...
ERROR:  new row for relation "members" violates check constraint "left_after_joined"
...
```

Two subtle rules:

- `CHECK` **passes when its expression is NULL** — it fails only on *false* (three-valued logic, chapter 2). Hence `left_on IS NULL` is allowed implicitly, and a required column also needs `NOT NULL`.
- `UNIQUE` **allows several NULLs** in PostgreSQL, MySQL, SQLite and Oracle (NULL ≠ NULL). SQL Server allows only **one** (use a filtered unique index — over rows matching a `WHERE` — for "unique when present"). PostgreSQL 15 added `UNIQUE NULLS NOT DISTINCT` (not run).

Foreign keys and referential actions

A foreign key keeps **referential integrity** (no child points to a missing parent). What happens when the *parent* is deleted or re-keyed is chosen with `ON DELETE` / `ON UPDATE`:

Action	When the referenced parent row is deleted (or its key updated)
`NO ACTION` (default)	error if children still reference it (checked at the end of the statement; can be deferred)
`RESTRICT`	error, checked immediately (can't be deferred)
`CASCADE`	delete (or update) the child rows too
`SET NULL`	set the child's FK column(s) to NULL
`SET DEFAULT`	set the child's FK column(s) to their default

```
CREATE TABLE clubs (
    club_id  integer PRIMARY KEY,
    name     text NOT NULL UNIQUE
);
CREATE TABLE club_memberships (
    club_id    integer REFERENCES clubs(club_id)     ON DELETE CASCADE,
    member_id  integer REFERENCES members(member_id) ON DELETE RESTRICT,
    role       text NOT NULL DEFAULT 'member',
    PRIMARY KEY (club_id, member_id)                                  -- composite key
);
CREATE TABLE club_events (
    event_id   integer PRIMARY KEY,
    club_id    integer REFERENCES clubs ON DELETE SET NULL,           -- "REFERENCES clubs" = its PK
    title      text NOT NULL
);
INSERT INTO clubs VALUES (10, 'Chess'), (20, 'Hiking');
INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1), (10, 2), (20, 1);
INSERT INTO club_events VALUES (100, 10, 'Blitz night'), (101, 20, 'Sunrise trek');

INSERT INTO club_memberships (club_id, member_id) VALUES (10, 1);       -- composite PK duplicate
INSERT INTO club_memberships (club_id, member_id) VALUES (30, 1);       -- no club 30
DELETE FROM members WHERE member_id = 2;                                  -- RESTRICT
```

Output:

```
ERROR:  duplicate key value violates unique constraint "club_memberships_pkey"
...
ERROR:  insert or update on table "club_memberships" violates foreign key constraint
"club_memberships_club_id_fkey"
DETAIL:  Key (club_id)=(30) is not present in table "clubs".
ERROR:  update or delete on table "members" violates foreign key constraint "club_memberships_member_id_fkey" on
table "club_memberships"
DETAIL:  Key (member_id)=(2) is still referenced from table "club_memberships".
```

Deleting Chess triggers `CASCADE` (memberships go) and `SET NULL` (its event stays, orphaned on purpose):

```
DELETE FROM clubs WHERE club_id = 10;
SELECT * FROM club_memberships ORDER BY club_id, member_id;
SELECT * FROM club_events ORDER BY event_id;
```

Output:

```
 club_id | member_id |  role
---------+-----------+--------
      20 |         1 | member
(1 row)

 event_id | club_id |    title
----------+---------+--------------
      100 |  (null) | Blitz night
      101 |      20 | Sunrise trek
(2 rows)
```

Choosing: `CASCADE` for rows *part of* the parent (order lines, memberships); `RESTRICT` / `NO ACTION` for things that must not vanish silently (a customer with orders); `SET NULL` when the child should survive unlinked. Constraints are listed in `pg_constraint` (`contype` `p` / `u` / `f` / `c`).

Deferrable constraints. Normally checked after each statement; `DEFERRABLE INITIALLY DEFERRED` postpones to `COMMIT`, allowing temporary inconsistency — e.g. two rows referencing each other:

```
CREATE TABLE people (
    person_id  integer PRIMARY KEY,
    partner_id integer REFERENCES people DEFERRABLE INITIALLY DEFERRED
);
BEGIN;
INSERT INTO people VALUES (1, 2);   -- person 2 doesn't exist yet: allowed for now
INSERT INTO people VALUES (2, 1);
COMMIT;                             -- checked here: both exist
```

Both rows are committed. (`UNIQUE`, `PRIMARY KEY`, `REFERENCES` and `EXCLUDE` can be deferrable in PostgreSQL; `CHECK` and `NOT NULL` cannot.)

Dialect notes

(InnoDB is MySQL's default **storage engine**, the component storing rows; `PRAGMA` is SQLite's settings command.)

	PostgreSQL	MySQL (InnoDB)	SQL Server	SQLite
`CHECK` enforced	yes	since 8.0.16 (parsed but ignored before)	yes	yes
FK enforced	yes	yes	yes	only after `PRAGMA foreign_keys = ON`
Index auto-created on FK column	**no** — create one yourself	yes	no	no
Multiple NULLs in `UNIQUE`	yes	yes	no (one)	yes
Deferrable constraints	yes	no	no	FKs only

When not to rely on constraints alone

Constraints check single rows (and FK existence). Multi-row rules ("at most 5 members per club") need triggers (functions run automatically on row changes), `EXCLUDE` constraints or careful transactions (chapter 13, chapter 14).

Common mistakes

- **No index on the FK column**: in PostgreSQL each parent delete then scans the whole child table.
- `ON DELETE CASCADE` **everywhere**: one delete can wipe out large parts of the database.
- **Forgetting** `NOT NULL` next to `CHECK` or `UNIQUE` when a value is required.
- **A changeable natural key** (email, phone) as the PK — every FK must change with it. Use a surrogate PK plus `UNIQUE` on the natural key.

In the interview

30-second answer: "A candidate key is a minimal set of columns that identifies a row; the primary key is the chosen one — unique, not null, one per table; the rest are alternate keys enforced with `UNIQUE` (NULLs allowed). A composite key combines columns. A foreign key references another table's key, and `ON DELETE CASCADE / SET NULL / RESTRICT` decides what happens to children. `CHECK`, `NOT NULL` and `DEFAULT` complete the set."

- *Primary key vs unique key?* → PK: one per table, never NULL, the row's identity (often the clustered index — the one that stores the rows themselves in key order, 9.7 — in SQL Server/MySQL). UNIQUE: many per table, NULLs allowed.
- *Natural or surrogate PK?* → Usually a surrogate (`bigint` identity or UUID) for stability, plus `UNIQUE` on the natural key to keep the business rule.

Self-check

1. *Can a table have two primary keys?* → No; one (possibly composite), any number of unique keys.
2. *Is* `{member_id, email}` *a candidate key?* → No — a super key, not minimal (`member_id` alone is unique).

9.4 Generated keys: identity, sequences, serial, UUID, generated columns

What it is

Most tables use a generated **surrogate key**. PostgreSQL generates numbers with a **sequence** — an object that hands out increasing values (`nextval`); an **identity column** (SQL standard, PostgreSQL 10+) is a column tied to its own sequence.

Syntax	Meaning
`GENERATED ALWAYS AS IDENTITY`	database always generates the value; supplying one is an error (unless `OVERRIDING SYSTEM VALUE`)
`GENERATED BY DEFAULT AS IDENTITY`	generated when you don't supply one; you may supply your own
`serial` / `bigserial`	older PostgreSQL shorthand: integer column + sequence + `DEFAULT nextval(...)`
`uuid DEFAULT gen_random_uuid()`	random 128-bit id — a UUID, *universally unique identifier* (built in since PostgreSQL 13)
`GENERATED ALWAYS AS (expr) STORED`	a **generated column**: computed from other columns on every write (PostgreSQL 12+)

```sql
-- code/04_identity_and_sequences.sql
CREATE TABLE tickets (
    ticket_id  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    subject    text NOT NULL,
    price      numeric(8,2) NOT NULL,
    qty        integer NOT NULL DEFAULT 1,
    total      numeric(10,2) GENERATED ALWAYS AS (price * qty) STORED
);
INSERT INTO tickets (subject, price) VALUES ('Printer jam', 10);
INSERT INTO tickets (subject, price, qty) VALUES ('VPN access', 5, 3), ('New laptop', 1200, 1)
RETURNING ticket_id, total;
```

Output:

```
 ticket_id |  total
-----------+---------
         2 |   15.00
         3 | 1200.00
(2 rows)
```

```sql
INSERT INTO tickets (ticket_id, subject, price) VALUES (99, 'Manual id', 1);
INSERT INTO tickets (subject, price, total) VALUES ('Fake total', 1, 500);
```

Output:

```
ERROR:  cannot insert a non-DEFAULT value into column "ticket_id"
DETAIL:  Column "ticket_id" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.
ERROR:  cannot insert a non-DEFAULT value into column "total"
DETAIL:  Column "total" is a generated column.
```

Sequences never go backwards — gaps are normal:

```sql
INSERT INTO tickets (ticket_id, subject, price) OVERRIDING SYSTEM VALUE VALUES (99, 'Manual id', 1);
BEGIN;
INSERT INTO tickets (subject, price) VALUES ('Will be rolled back', 1);
ROLLBACK;
INSERT INTO tickets (subject, price) VALUES ('After rollback', 2);
SELECT ticket_id, subject, total FROM tickets ORDER BY ticket_id;
```

Output:

```
 ticket_id |    subject     |  total
-----------+----------------+---------
         1 | Printer jam    |   10.00
 ...
         3 | New laptop     | 1200.00
         5 | After rollback |    2.00
        99 | Manual id      |    1.00
(5 rows)
```

Number 4 went to the rolled-back insert and is never reused: sequences are deliberately **not** transactional, so concurrent inserts never wait for each other. Never assume gap-free ids (legal invoice numbers need separate generation under a **lock** — a claim that makes other sessions wait).

Why identity beats `serial`: `serial` is just a default, so a manual id doesn't advance the sequence and a later one collides:

```
CREATE TABLE legacy (id serial PRIMARY KEY, label text);
INSERT INTO legacy (label) VALUES ('a'), ('b');       -- ids 1, 2
INSERT INTO legacy (id, label) VALUES (3, 'manual');  -- allowed
INSERT INTO legacy (label) VALUES ('c');              -- sequence gives 3 -> collision
```

Output:

```
ERROR:  duplicate key value violates unique constraint "legacy_pkey"
DETAIL:  Key (id)=(3) already exists.
```

`GENERATED ALWAYS` refuses the manual insert up front. (After loading your own ids, resync: `SELECT setval('legacy_id_seq', (SELECT max(id) FROM legacy));`.)

A standalone sequence (e.g. invoice numbers shared by several tables):

```
CREATE SEQUENCE invoice_no START 1000 INCREMENT 10;
SELECT nextval('invoice_no') AS first, nextval('invoice_no') AS second, currval('invoice_no') AS current;
-- 1000 | 1010 | 1010
```

`currval` returns the last value *this session* got from `nextval`.

UUIDs: `key_id uuid PRIMARY KEY DEFAULT gen_random_uuid()` gives 128-bit ids (36 characters as text), generated anywhere (even in the app) without coordination and not revealing row counts — but 16 bytes (vs 8 for `bigint`), and random values scatter index inserts. Time-ordered UUIDv7 (leading timestamp bits, so ids sort by creation) reduces that; PostgreSQL 18 adds `uuidv7()` (not run).

Dialect notes

	Auto-increment syntax
PostgreSQL	`GENERATED {ALWAYS | BY DEFAULT} AS IDENTITY` (10+), `serial` (legacy)
MySQL	`AUTO_INCREMENT`
SQL Server	`IDENTITY(1,1)`; `SEQUENCE` objects since 2012
SQLite	`INTEGER PRIMARY KEY` (alias of the rowid, SQLite's hidden row number); `AUTOINCREMENT` prevents reuse of deleted ids
Oracle	identity columns since 12c; sequences

Generated columns: PostgreSQL 12–17 `STORED` only; PostgreSQL 18 adds `VIRTUAL` (computed on read) and makes it the default when neither keyword is given (not run); MySQL 5.7+ supports both; SQL Server has "computed columns" (optionally `PERSISTED`).

9.5 ALTER: changing existing tables

What it is

`ALTER TABLE` changes an existing (usually populated) table's structure without recreating it.

Task	PostgreSQL
Add a column	`ALTER TABLE t ADD COLUMN c type [DEFAULT …] [NOT NULL];`
Drop a column	`ALTER TABLE t DROP COLUMN c;`
Rename a column / table	`ALTER TABLE t RENAME COLUMN a TO b;` / `ALTER TABLE t RENAME TO u;`
Change a type	`ALTER TABLE t ALTER COLUMN c TYPE newtype [USING expr];`
Set / drop a default	`ALTER TABLE t ALTER COLUMN c SET DEFAULT …;` / `DROP DEFAULT`
Set / drop NOT NULL	`ALTER TABLE t ALTER COLUMN c SET NOT NULL;` / `DROP NOT NULL`
Add / drop a constraint	`ALTER TABLE t ADD CONSTRAINT name …;` / `DROP CONSTRAINT name;`

How it works (with real data)

```
-- code/05_alter.sql
CREATE TABLE staff (id integer PRIMARY KEY, first_name text, last_name text);
INSERT INTO staff VALUES (1, 'Tara', 'Mendes'), (2, 'Omar', 'Mendes'), (3, 'Yuki', 'Tanaka');

ALTER TABLE staff ADD COLUMN salary integer;
ALTER TABLE staff ADD COLUMN country text NOT NULL DEFAULT 'US';
SELECT * FROM staff ORDER BY id;
```

Output:

```
 id | first_name | last_name | salary | country
----+------------+-----------+--------+---------
  1 | Tara       | Mendes    | (null) | US
  2 | Omar       | Mendes    | (null) | US
  3 | Yuki       | Tanaka    | (null) | US
(3 rows)
```

Existing rows get NULL or the default. Since PostgreSQL 11 a constant default is instant (no table rewrite — copying every row to a new file).

```
ALTER TABLE staff RENAME COLUMN first_name TO given_name;
ALTER TABLE staff RENAME TO team;

UPDATE team SET salary = 50000 + id * 1000;
ALTER TABLE team ALTER COLUMN salary TYPE numeric(10,2) USING salary::numeric(10,2);
ALTER TABLE team ALTER COLUMN last_name TYPE integer;                -- text -> integer?
```

Output:

```
ERROR:  column "last_name" cannot be cast automatically to type integer
HINT:  You might need to specify "USING last_name::integer".
```

`USING` says how to compute the new value; without it only automatic (implicit/assignment) casts — conversions PostgreSQL deems safe — are allowed.

`NOT NULL` on a column with NULLs fails — `ALTER TABLE team ADD COLUMN email text NOT NULL;` gives *ERROR: column "email" of relation "team" contains null values*. Add, backfill (fill existing rows), then constrain:

```
ALTER TABLE team ADD COLUMN email text;
UPDATE team SET email = lower(given_name) || '@corp.example';
ALTER TABLE team ALTER COLUMN email SET NOT NULL;
```

Live tables: `NOT VALID` then `VALIDATE`. A new `CHECK` or FK normally scans the table under a strong lock. Add it `NOT VALID` (enforced for new/changed rows only), fix old data, then `VALIDATE CONSTRAINT` (scans under a lighter lock):

```
ALTER TABLE team ADD CONSTRAINT team_email_uq UNIQUE (email);
INSERT INTO team (id, given_name, last_name, salary, email) VALUES (4, 'Leo', 'Brandt', -5, 'leo@corp.example');
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0);            -- fails: Leo's row
ALTER TABLE team ADD CONSTRAINT salary_positive CHECK (salary > 0) NOT VALID;  -- accepted
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                          -- fails until fixed
UPDATE team SET salary = 48000 WHERE id = 4;
ALTER TABLE team VALIDATE CONSTRAINT salary_positive;                          -- succeeds
```

Output:

```
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
ERROR:  check constraint "salary_positive" of relation "team" is violated by some row
```

(`UNIQUE` / `PRIMARY KEY` can't be `NOT VALID` ; build a unique index `CONCURRENTLY` — without blocking writes, 9.7 — then `ADD CONSTRAINT … UNIQUE USING INDEX` .)

Several changes in one statement (one lock, one pass):

```
ALTER TABLE team
    DROP COLUMN country,
    ADD COLUMN hired_on date,
    ALTER COLUMN last_name SET NOT NULL;
SELECT conname, pg_get_constraintdef(oid) AS definition
FROM pg_constraint WHERE conrelid = 'team'::regclass ORDER BY conname;
```

Output:

```
     conname     |           definition
-----------------+--------------------------------
 salary_positive | CHECK ((salary > (0)::numeric))
 staff_pkey      | PRIMARY KEY (id)
 team_email_uq   | UNIQUE (email)
(3 rows)
```

The PK is still `staff_pkey`: renaming a table doesn't rename its constraints or indexes.

Locks and production safety

Most `ALTER TABLE` forms take an **ACCESS EXCLUSIVE** lock — no reads or writes until done. Metadata-only changes (catalog edits that don't touch rows: nullable column, constant default, rename, drop column — which only hides it) hold it briefly; **type changes that rewrite the table** can hold it for minutes. Production practice: set `lock_timeout` (give up rather than wait forever), use `NOT VALID` + `VALIDATE` and `CREATE INDEX CONCURRENTLY`, and "expand → migrate → contract" (add the new column, backfill in batches of a few thousand rows, switch the app, drop the old). Locks: chapter 13.

Dialect notes

Task	PostgreSQL	MySQL	SQL Server
Change type	`ALTER COLUMN c TYPE t`	`MODIFY COLUMN c t`	`ALTER COLUMN c t`
Rename column	`RENAME COLUMN a TO b`	`RENAME COLUMN a TO b` (8.0) / `CHANGE a b t`	`EXEC sp_rename 't.a', 'b', 'COLUMN'`
Rename table	`ALTER TABLE t RENAME TO u`	`RENAME TABLE t TO u`	`EXEC sp_rename 't', 'u'`
DDL inside a transaction (rollback-able)	yes	no — implicit commit	yes (mostly)

Common mistakes

`NOT NULL` without a default or backfill (fails); a type change on a huge table in business hours (long exclusive lock); renaming columns that application SQL still uses (views follow renames automatically

in PostgreSQL, application code does not).

9.6 DROP and TRUNCATE (vs DELETE)

What they are

- `DROP` removes an object entirely — definition, data, indexes, constraints, triggers, permissions (`DROP TABLE/DATABASE/SCHEMA/VIEW/INDEX …`): throw the notebook away.
- `TRUNCATE` removes **all rows** but keeps the table (structure, indexes, grants): a fresh empty notebook. (`TABLE` in `TRUNCATE TABLE t` is optional in PostgreSQL and MySQL, required in SQL Server and Oracle.)
- `DELETE` (DML, 9.10) removes rows one by one, optionally filtered by `WHERE`: erasing lines.

How TRUNCATE works

```
-- code/06_drop_and_truncate.sql
CREATE TABLE audit_log (log_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY, msg text NOT NULL);
INSERT INTO audit_log (msg) SELECT 'event ' || g FROM generate_series(1, 5) g;

TRUNCATE audit_log;
SELECT count(*) AS rows_after_truncate FROM audit_log;                  -- 0
INSERT INTO audit_log (msg) VALUES ('after truncate') RETURNING log_id;   -- 6: identity continues
TRUNCATE audit_log RESTART IDENTITY;
INSERT INTO audit_log (msg) VALUES ('after restart') RETURNING log_id;    -- 1
```

Output (the three results): `0`, then `log_id = 6`, then `log_id = 1`.

In PostgreSQL, `TRUNCATE` is **transactional** — you can roll it back:

```
BEGIN;
TRUNCATE audit_log;
SELECT count(*) AS inside_tx FROM audit_log;       -- 0
ROLLBACK;
SELECT count(*) AS after_rollback FROM audit_log;  -- 1
```

A table referenced by a foreign key can't be truncated alone (orphaned children); `CASCADE` truncates the referencing tables too:

```
TRUNCATE customers;
BEGIN;
TRUNCATE customers CASCADE;
SELECT (SELECT count(*) FROM customers) AS customers,
       (SELECT count(*) FROM orders) AS orders,
       (SELECT count(*) FROM order_items) AS items;
ROLLBACK;
```

Output:

```
ERROR:  cannot truncate a table referenced in a foreign key constraint
DETAIL:  Table "orders" references "customers".
HINT:  Truncate table "orders" at the same time, or use TRUNCATE ... CASCADE.
NOTICE:  truncate cascades to table "orders"
NOTICE:  truncate cascades to table "order_items"
 customers | orders | items
-----------+--------+-------
         0 |      0 |     0
(1 row)
```

How DROP works

After `DROP TABLE audit_log;`, querying gives *relation "audit_log" does not exist* and a second `DROP` errors; `DROP TABLE IF EXISTS` only notices (use it in scripts). `DROP` refuses while objects depend on the table (default `RESTRICT`); `CASCADE` drops dependent views and FKs *pointing at* it (the child table and rows stay):

```
CREATE VIEW big_spenders AS SELECT customer_id FROM orders GROUP BY customer_id HAVING count(*) > 2;
DROP TABLE orders;
BEGIN;
DROP TABLE orders CASCADE;
SELECT count(*) AS order_items_still_there FROM order_items;
ROLLBACK;
```

Output:

```
ERROR:  cannot drop table orders because other objects depend on it
DETAIL:  constraint order_items_order_id_fkey on table order_items depends on table orders
view big_spenders depends on table orders
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
NOTICE:  drop cascades to 2 other objects
...
 order_items_still_there
-------------------------
                      22
(1 row)
```

`DROP DATABASE name;` removes a database (not while connected to it; in PostgreSQL not inside a transaction). `DROP SCHEMA s CASCADE;` removes a schema and its contents.

DELETE vs TRUNCATE vs DROP

	`DELETE FROM t [WHERE …]`	`TRUNCATE t`	`DROP TABLE t`
Kind	DML	DDL	DDL
Removes	chosen rows (or all)	all rows	rows **and** the table
`WHERE` possible	yes	no	no
Speed on big tables	slow: row by row, all logged (each change written to the transaction log)	fast: releases the data files	fast
Space returned to the OS	usually not: dead rows (old, deleted versions) are reused after `VACUUM` (PostgreSQL's cleanup process), the file rarely shrinks	yes	yes
Row-level `ON DELETE` triggers	fire	don't fire (PostgreSQL has `ON TRUNCATE` triggers)	n/a
FK checks	per row (or cascades)	refused if referenced, unless `CASCADE`	refused if referenced, unless `CASCADE`
Identity / auto-increment	continues	PostgreSQL: continues unless `RESTART IDENTITY` ; MySQL & SQL Server: reset	gone
Rollback	yes	inside a transaction in PostgreSQL & SQL Server; **not** in MySQL & Oracle (implicit commit)	same as `TRUNCATE`
Lock	row locks	exclusive table lock	exclusive table lock

Undoing `TRUNCATE` / `DROP` depends on the database: MySQL needs a backup (9.12); Oracle can undo `DROP TABLE` from its recycle bin (`FLASHBACK TABLE t TO BEFORE DROP` , if enabled) but not `TRUNCATE` . SQLite has no `TRUNCATE` ; `DELETE FROM t` without `WHERE` is optimized to do the same.

Common mistakes

- `DROP` / `TRUNCATE` on the wrong connection (production, the live system, instead of dev) — make production visually distinct; use reviewed migrations (versioned schema-change scripts).
- `… CASCADE` unread — try it in `BEGIN … ROLLBACK` first.
- `DELETE` to empty a 100-million-row table (hours, huge log) when `TRUNCATE` takes a second.
- Expecting `TRUNCATE` to be invisible to concurrent transactions: it isn't MVCC-safe (MVCC, *multi-version concurrency control*, gives each transaction a **snapshot** — a consistent view as of one moment; chapter 13), so an older-snapshot transaction reading afterwards sees it empty.

In the interview

> **30-second answer:** "`DELETE` is DML: selected rows, one at a time, fires triggers, keeps the table. `TRUNCATE` is DDL: all rows at once by deallocating storage — much faster, no `WHERE`, no delete triggers, often resets identity; rollback-able in PostgreSQL and SQL Server, not in MySQL/Oracle. `DROP` removes the table itself."

Self-check

1. *Empty a staging table (where incoming data lands before processing) fast and reset ids?* → `TRUNCATE staging RESTART IDENTITY;`
2. *`DROP TABLE orders CASCADE` — effect on `order_items`?* → Its FK constraint is dropped; table and rows remain.

9.7 Indexes (in brief)

An **index** is a separate structure (default a B-tree — balanced, always sorted) that finds rows by column values without reading the whole table. Only the DDL here; how indexes work, kinds, `EXPLAIN` (shows the query plan) and when they help: chapter 12.

```sql
-- code/07_indexes_brief.sql
CREATE INDEX idx_employees_last_name ON employees (last_name);
CREATE INDEX idx_orders_customer_placed ON orders (customer_id, placed_at DESC);   -- composite

-- A UNIQUE index can enforce rules a UNIQUE constraint can't, e.g. case-insensitive uniqueness:
CREATE UNIQUE INDEX idx_customers_email_ci ON customers (lower(trim(email)));
```

Output:

```
ERROR:  could not create unique index "idx_customers_email_ci"
DETAIL:  Key (lower(TRIM(BOTH FROM email)))=(anita.shah@mail.example) is duplicated.
```

The index found the duplicate Anita Shah; after `UPDATE customers SET email = NULL WHERE customer_id = 8` it succeeds.

PKs and unique constraints create indexes automatically (`pg_indexes`, the catalog view of indexes, lists `employees_pkey`, `employees_email_key`). Also: `DROP INDEX [IF EXISTS] name;`, and `CREATE/DROP INDEX CONCURRENTLY …`, which don't block writes on a live table (slower; not inside a transaction block).

Clustered vs non-clustered

	Clustered index	Non-clustered (secondary) index
What it is	the table's rows are **stored in the index's order** — the index *is* the table	a separate structure of (key → pointer to the row)
How many	at most one per table	many
SQL Server	`CREATE CLUSTERED INDEX ix ON t (col);` (the PK is clustered by default)	`CREATE [NONCLUSTERED] INDEX ix ON t (col);`
MySQL InnoDB	always the primary key (no PK → the first `UNIQUE NOT NULL` index, else a hidden row id)	every other index; it stores the PK value as the pointer
PostgreSQL	**none** — tables are unordered "heaps"; `CLUSTER t USING ix;` re-sorts the table once, but new rows aren't kept in order	every index is secondary

```
CLUSTER orders USING idx_orders_customer_placed;   -- one-time physical re-ordering in PostgreSQL
```

A clustered index makes range scans (every key between two values) fast — neighbouring keys are neighbouring rows — but a random key (random UUID) causes page splits on insert (a full page — the fixed-size storage block, 8 KB in PostgreSQL — split in two).

Cost: every index is maintained on inserts and updates of its columns (and cleaned after deletes) and takes disk. Index what queries filter, join and sort on — not every column.

9.8 INSERT

What it is

`INSERT` adds new rows to a table.

```
INSERT INTO table_name [(column1, column2, ...)]
VALUES (value1, value2, ...) [, (...), ...]
[ON CONFLICT ...]            -- 9.11
[RETURNING ...];             -- PostgreSQL: give back the inserted rows
-- or
INSERT INTO table_name [(columns)] SELECT ...;
```

How it works

1. The **column list** names supplied columns; omitted ones get their `DEFAULT`, else NULL (error if `NOT NULL` without default).
2. Without a list you supply **every** column in table order — fragile when columns are added.
3. If any row fails a type or constraint, the **whole statement** fails.

```sql
-- code/08_insert.sql
CREATE TABLE contacts (
    contact_id  integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name  text NOT NULL,
    last_name   text NOT NULL,
    age         integer CHECK (age >= 0),
    city        text DEFAULT 'Pune',
    created_at  timestamp NOT NULL DEFAULT '2025-06-01 09:00'
);

INSERT INTO contacts (first_name, last_name, age) VALUES ('Tara', 'Mendes', 28);

INSERT INTO contacts (first_name, last_name, age, city) VALUES      -- several rows at once
    ('Omar', 'Mendes', 31, 'Mumbai'),
    ('Yuki', 'Tanaka', NULL, DEFAULT);                              -- DEFAULT keyword

INSERT INTO contacts VALUES (DEFAULT, 'Leo', 'Brandt', 40, 'Delhi', '2025-06-02 10:00');  -- all columns

INSERT INTO contacts (first_name, last_name) VALUES ('Ines', 'Silva')
RETURNING contact_id, city, created_at;
```

Output (of the `RETURNING`):

```
 contact_id | city |     created_at
------------+------+---------------------
          5 | Pune | 2025-06-01 09:00:00
(1 row)
```

`RETURNING` hands back generated values (ids, defaults) in the same round trip (request plus reply) — no second query, no race (a **race condition** is a bug whose outcome depends on concurrent sessions' timing).

`INSERT … SELECT` copies rows produced by any query:

```sql
INSERT INTO contacts (first_name, last_name, city)
SELECT first_name, last_name, city FROM customers WHERE country = 'UK'
RETURNING contact_id, first_name, city;
```

Output:

```
 contact_id | first_name |  city
------------+------------+--------
          6 | Ben        | London
          7 | George     | London
(2 rows)
```

Errors worth recognising:

```sql
INSERT INTO contacts (first_name, last_name) VALUES ("Ana", "Lima");
INSERT INTO contacts (first_name, last_name) VALUES ('Ana', 'Lima', 22);
INSERT INTO contacts (first_name, last_name, age) VALUES ('Ok', 'Row', 20), ('Bad', 'Row', -1);
SELECT count(*) AS still_7 FROM contacts;
```

Output:

```
ERROR:  column "Ana" does not exist
ERROR:  INSERT has more expressions than target columns
ERROR:  new row for relation "contacts" violates check constraint "contacts_age_check"
DETAIL:  Failing row contains (9, Bad, Row, -1, Pune, 2025-06-01 09:00:00).
 still_7
---------
       7
(1 row)
```

- In standard SQL (PostgreSQL, SQLite, SQL Server by default, Oracle) **double quotes name identifiers** (table, column names); strings use single quotes. MySQL accepts double-quoted strings, so its code often breaks elsewhere.

- The multi-row insert failed whole — 'Ok' was not kept (but consumed identity values 8 and 9: more gaps).

Bulk loading: multi-row `VALUES` or `INSERT … SELECT` beats one statement per row; for millions of rows use `COPY table FROM file` (psql: `\copy` for a client file), MySQL `LOAD DATA INFILE`, SQL Server `BULK INSERT` / `bcp`.

Dialect notes

	PostgreSQL	MySQL	SQL Server	SQLite
Multi-row `VALUES`	yes	yes	yes (max 1000 rows per statement)	yes
Get generated values back	`RETURNING`	`LAST_INSERT_ID()`	`OUTPUT inserted.*` / `SCOPE_IDENTITY()`	`RETURNING` (3.35+) / `last_insert_rowid()`
`INSERT … DEFAULT VALUES`	yes	`INSERT INTO t () VALUES ()`	yes	yes

9.9 UPDATE

What it is

`UPDATE` changes column values in existing rows.

```
UPDATE table_name
SET column1 = expr1, column2 = expr2, ...
[FROM other_tables]          -- PostgreSQL: use other tables to compute / filter
[WHERE condition]            -- which rows; WITHOUT it, EVERY row changes
[RETURNING ...];
```

How it works

`WHERE` picks rows (none = all); every `SET` expression reads the row's **old** values, then all assignments apply together; new rows must satisfy all constraints or nothing changes. PostgreSQL writes a new row version (MVCC — chapter 13); `VACUUM` later removes the old.

Raise Sales salaries by 10 %:

```
-- code/09_update.sql
UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 2
RETURNING emp_id, first_name, salary;
```

Output:

```
 emp_id | first_name |  salary
--------+------------+-----------
      6 | Daniel     | 165000.00
      7 | Priya      |  79200.00
 ...
(4 rows)
```

+10 % is `* 1.10`, +50 % `* 1.5`, doubling `* 2` — check the factor against the requirement.

All `SET` expressions see the old row:

```
UPDATE products
SET unit_price   = unit_price * 0.9,
    discontinued = unit_price * 0.9 < 40          -- uses the OLD unit_price
WHERE category = 'Furniture'
RETURNING name, unit_price, discontinued;
```

Output:

```
     name       | unit_price | discontinued
----------------+------------+-------------
 Standing Desk  |     449.10 | f
 Ergonomic Chair |     314.99 | f
 Desk Lamp      |      31.50 | t
(3 rows)
```

(In MySQL, assignments are applied left to right, so a later expression sees the *new* value of an earlier column — a real portability trap. PostgreSQL and SQL Server follow the standard.)

Different changes for different rows with `CASE`:

```
UPDATE orders
SET status = CASE WHEN shipped_at IS NULL THEN 'cancelled' ELSE 'delivered' END
WHERE status IN ('pending', 'shipped') AND placed_at < '2025-06-10'
RETURNING order_id, status;
```

Output:

```
 order_id |  status
----------+-----------
      107 | delivered
 ...
      113 | cancelled
(4 rows)
```

Using another table — PostgreSQL's `UPDATE … FROM` joins the target to other tables:

```
ALTER TABLE customers ADD COLUMN lifetime_orders integer NOT NULL DEFAULT 0;
UPDATE customers c
SET lifetime_orders = t.n
FROM (SELECT customer_id, count(*) AS n FROM orders GROUP BY customer_id) t
WHERE t.customer_id = c.customer_id;
SELECT customer_id, first_name, lifetime_orders FROM customers ORDER BY customer_id LIMIT 4;
```

Output:

```
 customer_id | first_name | lifetime_orders
-------------+------------+-----------------
           1 | Anita      |               3
           2 | Ben        |               2
 ...
(4 rows)
```

Each target row must match **at most one** `FROM` row, or PostgreSQL uses one unpredictably — hence aggregating first. Portable alternative: a correlated scalar subquery (one value, re-run per outer row) in `SET`:

```
UPDATE departments d
SET budget = (SELECT coalesce(sum(salary), 0) * 1.5 FROM employees e WHERE e.dept_id = d.dept_id)
WHERE d.dept_id = 5
RETURNING name, budget;                          -- Support | 96000.00
```

The classic accident — `UPDATE` without `WHERE` updates every row. Practise risky statements inside a transaction, check, then `COMMIT` or `ROLLBACK`:

```
BEGIN;
UPDATE employees SET salary = 1;
SELECT count(*) AS rows_now_paid_1 FROM employees WHERE salary = 1;
ROLLBACK;
SELECT count(*) AS rows_paid_1_after_rollback FROM employees WHERE salary = 1;
```

Output:

```
 rows_now_paid_1
-----------------
              15
(1 row)

 rows_paid_1_after_rollback
----------------------------
                           0
(1 row)
```

Also: run the `WHERE` as `SELECT count(*)` first; check psql's `UPDATE n`; use `RETURNING`; MySQL clients' `--safe-updates` refuses key-less updates. Constraints still apply (`dept_id = 99` fails the FK, `unit_price = -1` the `CHECK`).

9.10 DELETE

What it is

`DELETE` removes rows from a table.

```
DELETE FROM table_name
[USING other_tables]         -- PostgreSQL: join to other tables
[WHERE condition]            -- WITHOUT it, EVERY row is deleted
[RETURNING ...];
```

How it works

`WHERE` picks rows (none = all; the table remains). Deleting a parent with children fails or cascades per its FKs (9.3). PostgreSQL only marks rows dead; `VACUUM` reclaims space later.

```
-- code/10_delete.sql
DELETE FROM order_items
WHERE order_id = 105
RETURNING order_id, product_id, quantity;
```

Output:

```
 order_id | product_id | quantity
----------+------------+----------
      105 |          1 |        2
(1 row)
```

```
DELETE FROM customers WHERE customer_id = 1;
```

Output:

```
ERROR:  update or delete on table "customers" violates foreign key constraint "orders_customer_id_fkey" on table "orders"
DETAIL:  Key (customer_id)=(1) is still referenced from table "orders".
```

Deleting based on another table — PostgreSQL's `DELETE … USING`, or the portable subquery form:

```sql
DELETE FROM order_items oi
USING orders o
WHERE o.order_id = oi.order_id
  AND o.status = 'cancelled'
RETURNING oi.order_id, oi.product_id;          -- (111, 2)

DELETE FROM orders
WHERE status = 'cancelled'
  AND order_id NOT IN (SELECT order_id FROM order_items)
RETURNING order_id;                            -- 105, 111

DELETE FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
RETURNING customer_id, first_name;
```

Output of the last statement:

```
 customer_id | first_name
-------------+------------
           4 | Divya
           9 | Hana
          10 | Ivan
(3 rows)
```

(Divya's only order, cancelled, was just removed.) Children first, then parents, as FKs demand. (`NOT IN` is safe here only because `order_items.order_id` can't be NULL; prefer `NOT EXISTS` — chapter 6.)

Delete everything — safely practised:

```sql
BEGIN;
DELETE FROM order_items;
SELECT count(*) AS items_inside_tx FROM order_items;       -- 0
ROLLBACK;
SELECT count(*) AS items_after_rollback FROM order_items;  -- 20
```

Soft delete — mark rows instead of physically deleting:

```sql
ALTER TABLE products ADD COLUMN deleted_at timestamp;
UPDATE products SET deleted_at = '2025-06-30 12:00' WHERE name = 'Desk Lamp';
CREATE VIEW active_products AS SELECT * FROM products WHERE deleted_at IS NULL;
SELECT count(*) AS all_products, (SELECT count(*) FROM active_products) AS active FROM products;
```

Output:

```
 all_products | active
--------------+--------
            9 |      8
(1 row)
```

	Hard delete	Soft delete
Undo / audit	only from backups or audit tables	trivial: clear `deleted_at`
Queries	simple	every query must filter (use a view, or row-level security — per-user rules that hide rows)
Unique constraints	simple	a deleted row still blocks its email — use a partial unique index (PostgreSQL's filtered index) `WHERE deleted_at IS NULL`
Privacy law ("right to be forgotten")	satisfied	must still purge eventually

Millions of rows: delete in batches (`DELETE … WHERE id IN (SELECT id … LIMIT 10000)` repeated) to keep transactions and locks short; with partitioned tables (one logical table as several physical pieces, e.g. per month) `DROP` an old partition instantly (chapter 15).

Dialect notes: UPDATE / DELETE using another table

	UPDATE	DELETE
PostgreSQL	`UPDATE t SET … FROM s WHERE t.id = s.id`	`DELETE FROM t USING s WHERE t.id = s.id`
SQL Server	`UPDATE t SET … FROM t JOIN s ON …`	`DELETE t FROM t JOIN s ON …`
MySQL	`UPDATE t JOIN s ON … SET t.col = …`	`DELETE t FROM t JOIN s ON …`
SQLite	`UPDATE t SET … FROM s WHERE …` (3.33+)	subquery only (`WHERE id IN (SELECT …)`)

9.11 Upsert: INSERT … ON CONFLICT, and MERGE

What it is

An **upsert** is "insert, or if the key exists, update (or skip)". `SELECT`-then-`INSERT` / `UPDATE` in app code races: two sessions both see "not there" and both insert. PostgreSQL (9.5+) does it atomically (one indivisible step) with `INSERT … ON CONFLICT`.

How it works

```
INSERT INTO t (cols) VALUES (...)
ON CONFLICT (conflict_target)          -- the unique/PK column(s), or ON CONSTRAINT name
DO NOTHING                             -- skip rows that would conflict
-- or
DO UPDATE SET col = EXCLUDED.col, ...  -- EXCLUDED = the row you tried to insert
         [WHERE condition];            -- update only when this is true
```

Each row is inserted, or — if it would violate the conflict target's unique index — the `DO` action runs instead of an error; atomically, even under concurrency. In `DO UPDATE`, `t.col` (or alias) is the **existing** row, `EXCLUDED.col` the **proposed** one.

```
-- code/11_upsert.sql
CREATE TABLE inventory (
    sku         text PRIMARY KEY,
    qty         integer NOT NULL CHECK (qty >= 0),
    updated_at  timestamp NOT NULL
);
INSERT INTO inventory VALUES ('MOUSE-1', 10, '2025-06-01 08:00'), ('HUB-2', 4, '2025-06-01 08:00');

INSERT INTO inventory VALUES ('MOUSE-1', 99, '2025-06-02 08:00'), ('LAMP-3', 7, '2025-06-02 08:00')
ON CONFLICT (sku) DO NOTHING
RETURNING sku, qty;
```

Output:

```
  sku   | qty
--------+-----
 LAMP-3 |   7
(1 row)
```

MOUSE-1 already existed, so it was skipped silently (and isn't in `RETURNING`).

Insert or add to the existing stock:

```
INSERT INTO inventory AS i VALUES
    ('MOUSE-1', 5, '2025-06-03 08:00'),
    ('KEYB-4',  2, '2025-06-03 08:00')
ON CONFLICT (sku) DO UPDATE
SET qty        = i.qty + EXCLUDED.qty,
    updated_at = EXCLUDED.updated_at
RETURNING sku, qty, updated_at, (xmax <> 0) AS was_update;
```

Output:

```
   sku   | qty |     updated_at      | was_update
---------+-----+---------------------+------------
 MOUSE-1 |  15 | 2025-06-03 08:00:00 | t
 KEYB-4  |   2 | 2025-06-03 08:00:00 | f
(2 rows)
```

(`xmax`, a hidden system column, `<> 0` is a common trick to tell updated from inserted rows — an internal detail; don't build business logic on it.)

Conditional update — only if incoming data is newer (common when syncing several sources):

```
INSERT INTO inventory AS i VALUES ('HUB-2', 1, '2025-05-01 00:00')    -- older than what we have
ON CONFLICT (sku) DO UPDATE
SET qty = EXCLUDED.qty, updated_at = EXCLUDED.updated_at
WHERE EXCLUDED.updated_at > i.updated_at
RETURNING sku, qty;
```

Output:

```
 sku | qty
-----+-----
(0 rows)
```

Nothing changed: HUB-2 keeps qty 4 from 1 June. **Rules and errors:**

- The conflict target must match a unique index or constraint (`ON CONFLICT (sku)` or `ON CONFLICT ON CONSTRAINT inventory_pkey`). `DO NOTHING` may omit the target (then any unique violation is skipped).
- One statement can't affect the same row twice:

```
INSERT INTO inventory VALUES ('PEN-5', 1, '2025-06-04'), ('PEN-5', 2, '2025-06-04')
ON CONFLICT (sku) DO UPDATE SET qty = EXCLUDED.qty;
```

Output:

```
ERROR:  ON CONFLICT DO UPDATE command cannot affect row a second time
HINT:  Ensure that no rows proposed for insertion within the same command have duplicate constrained values.
```

De-duplicate the input first (e.g. `SELECT DISTINCT ON (sku) …`).

- Other constraint violations (a `CHECK`, a different unique index) still raise errors.

MERGE (PostgreSQL 15+, SQL Server, Oracle)

`MERGE` is the standard "compare source with target and insert, update or delete accordingly". PostgreSQL added it in **version 15** (SQL Server 2008, Oracle 9i); on this book's 14 it is a syntax error. Shape (not run):

```
MERGE INTO inventory AS t
USING incoming AS s ON t.sku = s.sku
WHEN MATCHED AND s.qty = 0 THEN DELETE
WHEN MATCHED THEN UPDATE SET qty = s.qty, updated_at = s.updated_at
WHEN NOT MATCHED THEN INSERT (sku, qty, updated_at) VALUES (s.sku, s.qty, s.updated_at);
```

	`INSERT … ON CONFLICT`	`MERGE`
Can delete	no	yes (`WHEN MATCHED … THEN DELETE`)
Needs a unique index	yes	no — any join condition
Safe against concurrent inserts of the same key	yes (that's its purpose)	not by itself: two sessions can both take the `NOT MATCHED` branch → unique violation; retry or lock
`RETURNING`	yes	PostgreSQL 17+ (which also added `WHEN NOT MATCHED BY SOURCE` for deleting target rows missing from the source)

Rule of thumb in PostgreSQL: **upserts of single rows/batches →** `ON CONFLICT`**; synchronising a table from a staging table with deletes →** `MERGE` (15+).

Dialect notes

	Upsert syntax
PostgreSQL	`INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … EXCLUDED.col` (9.5+); `MERGE` (15+)
MySQL	`INSERT … ON DUPLICATE KEY UPDATE col = VALUES(col)` (or row alias `AS new … new.col`, 8.0.19+); `INSERT IGNORE` ≈ do nothing
SQLite	`INSERT … ON CONFLICT(col) DO UPDATE SET col = excluded.col` (3.24+); `INSERT OR IGNORE` / `INSERT OR REPLACE`
SQL Server	`MERGE`, or `UPDATE` + `INSERT … WHERE NOT EXISTS` with locking hints
Oracle	`MERGE`

In the interview

> **30-second answer:** "In PostgreSQL, `INSERT … ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col` — atomic and concurrency-safe because it relies on the unique index; `DO NOTHING` skips duplicates. `MERGE` (PostgreSQL 15+, SQL Server, Oracle) is the standard multi-branch statement that can also delete, but doesn't by itself protect against concurrent inserts."

Self-check

1. *What does* `EXCLUDED` *refer to?* → The row that was proposed for insertion.
2. *What does* `ON CONFLICT` *need?* → A unique index/constraint matching the conflict target.

9.12 Backup and restore

What it is

A **backup** is a copy to rebuild from; a **restore** rebuilds. Backups cover what replication (live copies — **replicas** — on other servers; System Design Fundamentals) can't: a mistaken `DELETE` / `DROP` (replicas copy it instantly), corrupting bugs, ransomware (malware encrypting files for payment), losing a data centre. Strategy is set by:

- **RPO — recovery point objective**: how much recent data you can afford to lose (e.g. 5 minutes).
- **RTO — recovery time objective**: how long you can afford to be down while restoring (e.g. 1 hour).

Kinds of backups

Kind	What is copied	Tools (PostgreSQL)	Good for	Limits
Logical	SQL / an archive of schema + data	`pg_dump`, `pg_dumpall`	small/medium DBs, single tables, moving across versions	slow for huge DBs; restore rebuilds indexes
Physical	the database files themselves	`pg_basebackup`, file-system snapshots, pgBackRest, Barman	large DBs, fast full restore	whole cluster (in PostgreSQL: one server instance with all its databases) only; same major version (e.g. 16.x)
Continuous (PITR)	a base backup **plus** the write-ahead log (WAL) stream	WAL archiving + base backup	restore to *any moment* (just before the bad `DELETE`)	more setup and storage

> **Key term — WAL and PITR:** the **write-ahead log (WAL)** records every change *before* it reaches the tables (chapter 1). **Point-in-time recovery (PITR)** restores a **base backup** (full physical copy) then replays archived WAL up to any moment — e.g. one second before a bad `DELETE`.

Also: **full** (everything), **differential** (since the last full), **incremental** (since the last backup of any kind; PostgreSQL 17 added incremental `pg_basebackup`), **hot/online** (taken while in use — all of the above).

pg_dump and pg_restore, for real

`pg_dump` takes a **consistent** snapshot of one database while others keep working (one repeatable-read transaction — an isolation level seeing data as of its start; chapter 13). Formats:

Format flag	Output	Restore with	Notes
`-F p` (plain, default)	a `.sql` script	`psql -f`	readable, editable; replayed statement by statement (`--single-transaction` for all-or-nothing)
`-F c` (custom)	one compressed archive	`pg_restore`	pick objects to restore, reorder, parallel restore (`-j`)
`-F d` (directory)	a folder, one file per table	`pg_restore`	the only format that can **dump** in parallel (`-j`)
`-F t` (tar)	a tar archive (a standard Unix file bundle)	`pg_restore`	like custom, but no compression or parallel restore

A full round trip on the book's server (temporary directory, deleted afterwards):

```
# code/12_backup_restore.sh (key lines; CONN="-h /tmp -p 5544 -U postgres")
pg_dump $CONN -d shop -F p -f "$WORK/shop.sql"          # plain SQL script
pg_dump $CONN -d shop -F c -f "$WORK/shop.dump"         # custom archive
pg_dump $CONN -d shop -F d -j 2 -f "$WORK/shop_dir"     # directory format, 2 parallel jobs
pg_restore -l "$WORK/shop.dump"                         # list the archive's contents
psql $CONN -d shop -c "DROP TABLE order_items"          # the "disaster"
pg_restore $CONN -d shop -t order_items "$WORK/shop.dump"      # bring back one table
pg_restore $CONN -d shop_copy -j 2 "$WORK/shop.dump"    # full restore into a new, empty DB
psql $CONN -d shop_plain -f "$WORK/shop.sql"            # plain dumps are replayed by psql
pg_dump $CONN -d shop -t products --schema-only         # structure only (--data-only: rows only)
```

Output (run with `bash chapters/09-ddl-dml-and-constraints/code/12_backup_restore.sh` from the book root):

```
== 1. a database to protect
shop: 10 customers, 14 orders, 22 order_items
...
-- plain dump contains 8 COPY blocks (one per table with data)
...
== 4. disaster: someone drops a table
ERROR:  relation "order_items" does not exist
== 5. restore just that table (definition + data) from the archive
shop: 10 customers, 14 orders, 22 order_items
-- note: -t restores the table and its rows, not its PK/FK constraints:
0 constraints on order_items
== 6. full restore into a NEW database, 2 parallel jobs
shop_copy: 10 customers, 14 orders, 22 order_items
3 constraints on order_items
== 7. a plain dump is restored with psql
shop_plain: 10 customers, 14 orders, 22 order_items
== 8. schema-only and data-only dumps of one table
CREATE TABLE statements in schema-only dump: 1, in data-only dump: 0
...
```

1. A plain dump is SQL: `CREATE TABLE`s, a `COPY … FROM stdin` block per table (`stdin` , *standard input*: rows inline in the script), then constraints and indexes (added *after* the data — faster).
2. A custom archive has a **table of contents** (`pg_restore -l`); `-t` , `-n` (schema), `-L` (edited list) choose what to restore.
3. `-t` restores definition and rows **but not constraints or indexes** (separate entries) — re-add via `-L` or the DDL.

Options: `--clean --if-exists` (drop before recreating), `--create` (create the database), `--no-owner` (restoring as another user), `-j N` . **Roles and tablespaces** (users/groups, named disk locations) are cluster-wide — also run `pg_dumpall --globals-only` .

Backup and restore in other databases

	Full backup	Restore	Notes
PostgreSQL	`pg_dump` (logical), `pg_basebackup` + WAL (physical, PITR)	`pg_restore` / `psql` ; restore base backup + replay WAL to a target time	command-line tools, not SQL statements
MySQL	`mysqldump` , MySQL Shell dump utilities (logical); Percona XtraBackup / MySQL Enterprise Backup (physical)	`mysql < dump.sql` ; binary logs (MySQL's log of every change) for point-in-time	
SQL Server	`BACKUP DATABASE db TO DISK = '…\db.bak'` (also `WITH DIFFERENTIAL` , `BACKUP LOG` , `TO URL` for cloud storage)	`RESTORE DATABASE db FROM DISK = '…' [WITH NORECOVERY / RECOVERY, STOPAT = …]`	backup and restore are T-SQL (SQL Server's SQL dialect) statements; the others mostly use separate tools
SQLite	`.backup file` in the shell, `VACUUM INTO 'file'`	copy the file back / `.restore`	the database is one file

Two tutorial errors corrected (SQL Server, not run):

- `BACKUP DATABASE` has no `WITH SCHEDULE = …` ; schedule via SQL Server Agent jobs or maintenance plans (elsewhere cron/systemd timers — Linux job schedulers — or a backup tool).
- There is no `RESTORE TABLE` : restore under **another database name** (`RESTORE DATABASE db_old FROM DISK = … WITH MOVE …`) and copy the table across — like `pg_restore -t` or `shop_copy` .

Best practices

- **3-2-1 rule**: 3 copies, on 2 kinds of storage, 1 off-site.
- **Test restores regularly** — an untested backup is a hope; automate a restore to a scratch server with sanity queries (row counts).

- **Encrypt** backups and restrict who can read them.
- **Match RPO/RTO**: nightly `pg_dump` alone risks 24 h of loss; PITR gets it to seconds.
- **Replication is not a backup** — replicas faithfully copy `DROP TABLE` (chapter 15).
- Managed databases (Amazon RDS, Cloud SQL, Azure) provide automated snapshots + PITR within a retention window (how long backups are kept).

In the interview

> **30-second answer:** "Logical backups (`pg_dump`) export schema and data — flexible, single tables, portable across versions, slow for huge databases. Physical backups copy the data files; with archived WAL they allow point-in-time recovery. Choose by RPO/RTO, keep 3-2-1 copies, encrypt, test restores — and replication doesn't protect against a bad `DELETE` ."

- *`DELETE` without `WHERE` at 14:03 — recovery?* → PITR: restore the last base backup to a new server, replay WAL to 14:02:59, copy the lost rows back (or fail over — switch the app to it).
- *Does pg_dump block a busy database?* → No (consistent snapshot), but its light locks block DDL like `DROP` / `ALTER` on dumped tables meanwhile.

Self-check

1. *Which formats does `pg_restore` read?* → custom, directory, tar (plain is replayed with `psql`).
2. *What does `pg_dumpall --globals-only` add?* → Roles and tablespaces.

Chapter 9 TL;DR

- **Types** validate as well as store: `numeric` for money, `integer` / `bigint` for ids, `text` , `timestamptz` , `date` , `boolean` .
- **`CREATE`** databases, schemas, tables (`IF NOT EXISTS` , `AS SELECT` = data without constraints, `LIKE … INCLUDING ALL` = structure, `TEMP`).
- **Keys**: super ⊇ candidate → one primary (unique + not null) + alternate keys (`UNIQUE`); composite keys; foreign keys with `ON DELETE CASCADE | SET NULL | RESTRICT | NO ACTION` . `CHECK` rejects only false; `UNIQUE` allows several NULLs (one in SQL Server). Name constraints; index FK columns.
- **Generated keys**: `GENERATED ALWAYS AS IDENTITY` (not `serial`), sequences have gaps, UUIDs for distributed ids, `GENERATED … STORED` for derived columns.
- **`ALTER`** : add/drop/rename columns, `TYPE … USING` , `SET NOT NULL` after backfilling, `NOT VALID` + `VALIDATE` on big tables; watch exclusive locks.
- **`DROP`** removes the object, **`TRUNCATE`** empties it fast (transactional in PostgreSQL, `RESTART IDENTITY` , `CASCADE`), **`DELETE`** removes chosen rows.
- **Indexes**: `CREATE [UNIQUE] INDEX [CONCURRENTLY]` ; clustered vs non-clustered; details in chapter 12.
- **DML**: `INSERT` (list columns, multi-row, `INSERT … SELECT`), `UPDATE` (`SET` sees old values, `FROM`), `DELETE` (`USING` , children first), all with `RETURNING` . Practise destructive statements in `BEGIN … ROLLBACK` .
- **Upsert**: `INSERT … ON CONFLICT … DO NOTHING / DO UPDATE SET … = EXCLUDED.…` ; `MERGE` needs PostgreSQL 15+.
- **Backups**: `pg_dump` (plain/custom/directory) + `pg_restore` (selective, parallel); physical + WAL for PITR; RPO/RTO, 3-2-1, test restores.

Next: chapter 10 — data modeling and ER diagrams, which decides *which* tables and keys to create in the first place.